

Executive Summary

We conducted a comprehensive analysis to design an **automated pipeline** for fixing and restructuring the research paper (`Paper_V3`) into a publication-ready format. The solution leverages **Python-based tools** (Pandoc, Python libraries, Mermaid CLI, etc.) and follows IEEE/ICDAR guidelines to enforce correct structure and formatting. Key steps include auditing and reordering sections (Title, Abstract, Keywords, Introduction, Related Work, Methods, Experiments, Results, Discussion, Conclusion, Ethics, Appendices), enforcing an IMRaD/IEEE layout, rendering diagrams, converting math, standardizing references, and generating final DOCX/PDF output. We integrate tools such as **Pandoc** (for Markdown→DOCX/PDF), **py pandoc**, **python-docx**, **python-docx-template**, **pandoc-crossref**, **pandoc-citeproc** (for IEEE citation style), **mermaid-cli** (diagram conversion), **latex2mathml** (math conversion), **markdown-it-py/mdformat** (linting), and continuous-integration workflows (GitHub Actions). Robust Python scripts parse and validate the manuscript structure, fix numbering, and polish tables. A **CI/CD pipeline** (e.g. GitHub Actions) runs markdown linting, diagram rendering, pandoc conversion, PDF generation, and publishes artifacts. We provide example YAML configurations, pytest-based QA checks (e.g. asserting no raw Mermaid/LaTeX remains, verifying figures and references), and a migration plan (branching, safe scripts, backups). We also outline a timeline (in hours) and a final checklist for human verification (covering grammar, format, and compliance). Tables compare tool features, and mermaid charts illustrate the automated workflow and project timeline. All recommendations align with IEEE Access and ICDAR formatting requirements [17†L67-L75] [18†L7-L16] , ensuring the final manuscript adheres to publication standards.

1. Prioritized Structural Fixes

We first ensure the manuscript follows **IEEE/ICDAR structure** (Title, Abstract, Keywords, Introduction, Related Work, Methods, Experiments, Results, Discussion, Conclusion, Ethics, Acknowledgments, References, Appendices). Key fixes include:

- **Title/Abstract/Keywords:** Verify title is concise and descriptive. Ensure the abstract succinctly summarizes the problem, methods, results, and significance (no citations or math) [17†L69-L77] . Provide 3–5 keywords after the abstract.
- **Section Ordering (IMRaD):** Reorder sections into a logical flow: start with *Introduction* (with problem context and literature review) [18†L7-L16] , then *Related Work* (if separate), then *Methods/System* (describe the proposed framework and synthetic dataset generation), *Experimental Setup*, *Results* (present evaluation metrics), *Discussion/Limitations*, and *Conclusion/Future Work*.
- **Contributions Statement:** Ensure a clear “Contributions” paragraph (often at end of Introduction) listing bullet-point contributions. IEEE journals often prefer authors to highlight novelty explicitly.
- **Related Work Placement:** Place Related Work early (usually Section II) or integrate into the Introduction, per IEEE norms [18†L7-L16] . Summarize how this work differs from benchmarks (SROIE, FUNSD, etc.).
- **Methods/Experiments/Results:** Organize the 360-specimen evaluation as “Experiments” or “Results” sections. Use subheadings (e.g. “7.1 Baselines and Metrics”) for clarity.

- **Discussion & Limitations:** Add a dedicated *Discussion* or *Analysis* section interpreting results, and explicitly discuss limitations or assumptions (e.g. synthetic vs. real data, single-model baseline scope) 【18†L7-L16】 .
- **Ethics/Privacy:** Insert required statements about FERPA/GDPR or IRB if not already present, as specified by IEEE/ICDAR guidelines. Often these go either in Acknowledgments or as footnotes following author info.
- **Appendices:** If additional details (e.g. full error taxonomy, sample images) exist, move them to numbered appendices after References.
- **Numbering and Styling:** Use IEEE numbering (Roman numerals for sections, capital letters for subsections) and two-column format as per IEEE template 【13†L84-L92】 . Ensure all headings and captions follow IEEE capitalization.

By aligning to these conventions, the paper will match examples of accepted IEEE journal submissions (e.g. IEEE Access) and ICDAR/ICDAR LNCS styles 【17†L69-L77】 【15†L96-L104】 .

2. Tools and Libraries

We recommend **modern, mature Python-based tools** and utilities to automate conversion and formatting. Below is a comparison of key tools:

Tool/Library	Purpose	Pros	Cons	License/ Maturity
Pandoc (CLI)	Universal converter (Markdown ↔ DOCX/PDF/HTML/etc.)	Highly flexible; extensive format support; supports citations (via <code>--citeproc</code>), cross-references (via filters) 【43†L218-L221】 ; active community.	CLI has learning curve; complex options for custom templates.	Open source (GPL); actively maintained.
py pandoc	Python wrapper for Pandoc	Scriptable Pandoc calls; integrates into Python scripts.	Requires Pandoc installed separately.	MIT; stable.
python-docx	Read/write Word (.docx) documents in Python	Fine-grained control of Word elements; create/edit tables, styles.	Does <i>not</i> support Word equations or PDF output; no native PDF.	MIT; stable (v0.8+).
python-docx-template	Templating engine for .docx (Jinja2-based)	Populate .docx templates with variables and loops.	Requires pre-made template; not needed for conversion, more for generation.	MIT; popular.

Tool/Library	Purpose	Pros	Cons	License/ Maturity
pandoc-crossref	Pandoc filter for figures/tables/references crossrefs	Automates numbering of figures/tables/equations and refs.	Setup needed; learning for syntax ([@fig:label]).	MIT; widely used.
pandoc-citeproc (or <code>--citeproc</code>)	Citation processor (CSL)	Formats citations/bibliography from BibTeX/CSL; supports IEEE, etc [43†L218-L221] .	Must supply correct CSL file (e.g. IEEE); JSON intermediate.	Open source; integrated in Pandoc.
markdown-it-py	Python Markdown parser	RFC-compliant; extensible; supports plugins (e.g. mermaid).	Code-heavy to use; less “out-of-box” than CLI tools.	MIT; actively developed.
mdformat	Markdown formatter/linter (Python)	Enforces consistent style (indentation, ATX headings, list “1.” etc.) [47†L31-L39] ; easy CLI (<code>mdformat file.md</code>).	Opinionated style (may conflict with some custom MD).	BSD; stable.
mermaid-cli	Converts Mermaid diagrams (<code>.mmd</code>) to SVG/PNG/PDF	Official CLI for mermaid; outputs high-res vector/PNG [49†L252-L254] .	Requires Node.js; diagrams must be valid Mermaid.	MIT; maintained by Mermaid team.
latex2mathml	LaTeX → MathML converter	Converts LaTeX math to MathML (for Word) or XML.	May not cover all macro cases; install via pip.	BSD; mature.
pylatexenc	Parse LaTeX expressions to Unicode/math text	Can convert LaTeX math to text or Unicode symbols.	Limited for complex math; easier for simple formulas.	BSD.
markdown-mermaid ^{*</sup>}	(Pandoc filter) embed mermaid images	Automatically finds <code>``mermaid</code> blocks and creates images.	Additional setup; still under development.	Open-source.

Tool/Library	Purpose	Pros	Cons	License/ Maturity
nbconvert	Jupyter Notebook ↔ Markdown/PDF	Useful if parts of manuscript came from notebooks.	Overkill if not using notebooks; requires LaTeX/Pandoc.	Open source (Nbconvert).
citeproc/pybtex	Bibliography processing (Python)	Python libraries to parse <code>.bib</code> or <code>.bibtex</code> ; can format references.	Manual coding needed for conversion to IEEE numbering.	MIT/Apache; used in many projects.
markdownlint / mdl	Markdown style checker (lint)	Enforces markdown style rules (line length, headings).	Setup/config required; just for syntax, not structure.	Unlicense/ others; popular CLI.

There exist Pandoc filters (e.g. `pandoc-mermaid-filter`) or Mermaid CLI usage to handle diagrams.

Pros and Cons: Pandoc is the central engine, handling most conversions including math (to Office MathML [36†L54-L57]) and citations. Python libraries allow fine-tuning (e.g. `python-docx` for table formatting). Mermaid CLI reliably generates diagrams. Tools like `mdformat/markdownlint` ensure Markdown quality and can auto-fix simple list/heading issues [47†L31-L39]. However, complex layout (two-column formatting, specific styles) often requires final manual adjustment in Word.

Install tools via `pip` (e.g. `pip install pandoc py pandoc python-docx mdformat latex2mathml pylatexenc`) or system packages (Pandoc, Node.js for `mermaid-cli`). For Pandoc, use the official installer or `apt-get`.

3. Pipeline Design & Code Snippets

We propose a **modular Python pipeline** with distinct stages. Key tasks and example code excerpts:

- **Parse/Validate Manuscript Structure:** Use `markdown-it-py` or simple regex to verify sections. For example, to ensure all major headings exist:

```
import re
text = open("Paper_V3.md").read()
required_sections = ["Introduction", "Method", "Results", "Discussion",
"Conclusion"]
for sec in required_sections:
    assert re.search(rf"^\s*{sec}", text, re.MULTILINE), f"Missing section: {sec}"
```

Or using `markdown_it` to parse headings:

```
from markdown_it import MarkdownIt
md = MarkdownIt()
tokens = md.parse(text)
headings = [t.content for t in tokens if t.type == 'heading_open']
```

- **Lint & Format Markdown:** Run `mdformat` to auto-fix list numbering, heading style, spacing:

```
mdformat --wrap 0 Paper_V3.md # wrap=0 keeps semantic line breaks
```

This enforces ATX headings and consistent lists [\[47†L37-L45\]](#) .

- **Convert Mermaid to Images:** Extract each `mermaid` block and render with Mermaid CLI. For example:

```
import subprocess, re
# Find mermaid blocks in markdown
mermaids = re.findall(r'```mermaid\s+(.*?)```', text, re.DOTALL)
for i, diagram in enumerate(mermaids, start=1):
    with open(f"fig{i}.mmd", "w") as f: f.write(diagram)
    subprocess.run(["mmdc", "-i", f"fig{i}.mmd", "-o", f"figure{i}.png", "-b", "transparent"])
    # Replace block with image link
    text = text.replace(diagram, f"![[Figure {i}]](figure{i}.png)")
open("Paper_V3_with_figures.md", "w").write(text)
```

Alternatively, use Pandoc filter `pandoc-mermaid` to automate this during conversion [\[49†L301-L309\]](#) .

- **Convert LaTeX Math to Word Equations:** Pandoc inherently converts TeX math to Word OMML (math objects) [\[36†L54-L57\]](#) . For example:

```
pandoc -f markdown -t docx --mathml -o Paper_V3_IEEE.docx
Paper_V3_with_figures.md
```

The `--mathml` flag ensures math is written as MathML (Office Math) [\[36†L54-L57\]](#) . If any raw LaTeX remains, we can preprocess:

```
from latex2mathml.converter import convert
text = re.sub(r"\$(.*?)\$", lambda m: convert(m.group(1)), text)
```

(Or use `pylatexenc` to convert simpler expressions to Unicode). The above Pandoc conversion typically handles most math properly.

- **Normalize Headings and Numbering:** Ensure top-level sections start at I. Pandoc's `-N` (number sections) can auto-number sections. Example:

```
pandoc -s --number-sections -o Paper_V3_numbered.md
Paper_V3_with_figures.md
```

For list numbering, `mdformat` already fixes to start at "1." [47†L45-L49]. Pandoc filters or manual post-editing with `python-docx` can fix any persisting sequence issues.

- **Professionalize Tables:** Use either a Word template or Python to style. With Pandoc, one can supply a reference DOCX (`--reference-doc`) that contains styled table formats. Or after conversion, adjust via `python-docx`:

```
from docx import Document
doc = Document("Paper_V3_IEEE.docx")
for table in doc.tables:
    table.style = 'Table Grid'
    # Center-align header row
    for cell in table.rows[0].cells:
        cell.paragraphs[0].runs[0].font.bold = True
        cell.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
doc.save("Paper_V3_IEEE.docx")
```

This ensures consistent borders and header emphasis.

- **Enforce IEEE Reference Style:** Convert the BibTeX (`references.bib`) to numbered IEEE format using Pandoc citeproc:

```
pandoc --citeproc --bibliography=refs.bib --csl=ieee.csl -s -o
Paper_V3_IEEE.docx Paper_V3_IEEE.md
```

or in Python with `pypandoc`:

```
import pypandoc
pypandoc.convert_file("Paper_V3_IEEE.md", 'docx',
    outputfile="Paper_V3_IEEE.docx",
    extra_args=['--citeproc', '--csl=ieee.csl'])
```

This produces “[1], [2], ...” citations and formats the bibliography in IEEE style **【43†L218-L221】** .

- **Final Conversion to PDF:** Finally, generate a PDF from the DOCX, either via Word automation or by converting MD with LaTeX:

```
pandoc Paper_V3_IEEE.md -o Paper_V3_IEEE.pdf --pdf-engine=xelatex --toc
```

or simply export from Microsoft Word (to ensure styling).

```
graph LR
    A[Lint Markdown (mdformat/markdownlint)] --> B[Render Mermaid Diagrams (mermaid-cli)]
    B --> C[Convert Markdown→DOCX (Pandoc with citeproc + crossref)]
    C --> D[Convert LaTeX Math to Word Equations (Pandoc MathML)]
    D --> E[Post-process DOCX (python-docx: style tables/heading)]
    E --> F[Generate PDF (Pandoc or Word)]
    F --> G[Publish artifacts]
```

4. CI/CD Workflow (GitHub Actions)

To automate these steps, we suggest a GitHub Actions workflow. An example `ci.yml` might look like:

```
name: Build Paper
on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build-paper:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repo
        uses: actions/checkout@v3

      - name: Set up Node.js (for mermaid-cli)
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - name: Install mermaid-cli
        run: npm install -g @mermaid-js/mermaid-cli

      - name: Set up Python
```

```

    uses: actions/setup-python@v4
    with:
      python-version: '3.x'

- name: Install Python tools
  run: |
    python -m pip install --upgrade pip
    pip install py pandoc python-docx mdformat latex2mathml pylatexenc
    bibtexparser pandocfilters

- name: Lint Markdown
  run: |
    mdformat --wrap 0 Paper_V3.md
    markdownlint Paper_V3.md

- name: Validate Structure (Python)
  run: python scripts/check_structure.py

- name: Render Diagrams
  run: python scripts/render_mermaid.py

- name: Convert to DOCX
  run: |
    pandoc Paper_V3_with_figures.md -o Paper_V3_IEEE.docx \
      --filter pandoc-crossref --citeproc --csl=ieee.csl --number-
sections

- name: Post-process DOCX
  run: python scripts/post_process_docx.py

- name: Convert to PDF
  run: pandoc Paper_V3_IEEE.md -o Paper_V3_IEEE.pdf --pdf-engine=xelatex

- name: Upload Artifacts
  uses: actions/upload-artifact@v3
  with:
    name: Paper_Documents
    path: |
      Paper_V3_IEEE.docx
      Paper_V3_IEEE.pdf

```

This workflow (inspired by [32]) checks out the code, sets up Node and Python, installs needed packages, lints the Markdown, runs custom Python scripts for structure check and diagram rendering, invokes Pandoc to produce DOCX and PDF, and then uploads the outputs as artifacts. Each step is documented with comments, and tools like `actions/checkout`, `actions/upload-artifact` are used as shown in [32] [32†L75-L84] [32†L86-L95] .

5. Quality Assurance Tests (pytest)

We implement automated tests to catch formatting issues. For example:

```
# test_qc.py
import re
from docx import Document

def test_no_raw_mermaid():
    text = open('Paper_V3.md').read()
    assert '`mermaid' not in text, "Unconverted Mermaid code found in markdown."

def test_no_latex_in_docx():
    doc = Document('Paper_V3_IEEE.docx')
    full_text = "\n".join(p.text for p in doc.paragraphs)
    assert '\\frac' not in full_text and '\\mathcal' not in full_text and '\\text{' not in full_text, "Raw LaTeX found in DOCX."

def test_figures_embedded():
    doc = Document('Paper_V3_IEEE.docx')
    # Ensure at least 2 figures embedded
    shapes = doc.inline_shapes
    assert len(shapes) >= 2, "Figures not properly embedded."

def test_references_numbered():
    doc = Document('Paper_V3_IEEE.docx')
    # Check references section paragraphs start with [#
    refs = [p.text for p in doc.paragraphs if re.match(r'^\\[\\d+\\]', p.text)]
    assert len(refs) > 0, "References not in IEEE numbered format."
```

These pytest tests check for (a) absence of raw Mermaid code, (b) no raw LaTeX (`\frac`, etc) in the Word text, (c) presence of embedded figure images, and (d) correctly numbered references (matching “[1]”, “[2]” etc.). Additional checks could verify table widths (using `python-docx` table dimensions) and that all required sections/headings are present. Running `pytest` in CI ensures any regressions are caught early.

6. Migration Plan (Patch)

To apply these changes to `Paper_V3.md` with minimal manual work, we propose:

1. **Backup:** Create a Git branch (`reformat-v1.0`) and tag the original commit (e.g. `v1.0-original`) before any edits.
2. **Automated Edits:** Run the Python scripts above (structure fixer, mermaid renderer) to generate a new Markdown + DOCX. For any non-automatable fixes (e.g. minor wording or figure caption tweaks), edit manually *on the generated files only*.

3. **Comparison:** Diff the original vs new files. Resolve any conflicts manually, ensuring no content was lost.
4. **Commit & Tag:** Commit changes with a message like “Apply formatting fixes (automated)”. Tag this as `v1.0-camera-ready`.
5. **Push:** Merge or push the branch to main after review, ensuring all generated files (`.docx`, `.pdf`, scripts, workflow YAML) are in the repo.

For example, a safe Python snippet might be:

```
import subprocess
subprocess.run(["git", "checkout", "-b", "reformat-v1.0"], check=True)
subprocess.run(["git", "tag", "v1.0-original"], check=True)
subprocess.run(["python", "scripts/render_mermaid.py"], check=True)
subprocess.run(["mdformat", "--wrap", "0", "Paper_V3.md"], check=True)
subprocess.run(["pandoc", "Paper_V3.md", "-o", "Paper_V3_IEEE.docx",
                "--citeproc", "--csl=ieee.csl", "--filter", "pandoc-crossref"],
                check=True)
subprocess.run(["python", "scripts/post_process_docx.py"], check=True)
subprocess.run(["git", "add", "Paper_V3_IEEE.docx"], check=True)
subprocess.run(["git", "commit", "-m", "Automated formatting fixes (Pandoc +
Python)"], check=True)
subprocess.run(["git", "tag", "v1.0-release"], check=True)
```

This ensures a reproducible, version-controlled transition. All script outputs and backups (tags) provide safety nets.

7. Risk Analysis & Fallbacks

- **Diagrams Fail:** If `mermaid-cli` fails on a diagram (syntax error), the fallback is manual drawing or using alternative tools (e.g. export from an online Mermaid editor). Keep the original text block as a last resort.
- **Math Conversion Issues:** Very complex LaTeX may not convert cleanly. If Pandoc’s auto conversion produces errors, fallback is to render equations as PNG (using MathJax or LaTeX) and embed those images in Word. Or manually recreate equations in Word’s editor.
- **Table Overflow:** If a table is too wide, we may need to manually split it or reduce font size. Automation can only do so much; in worst case, edit the DOCX.
- **Citation/Crossref Errors:** If the `.bib` file has syntax issues, Pandoc might misformat citations. Always validate the `.bib` with tools (e.g. `bibtexparser`). If automation fails, format references manually in Word as a last resort.
- **Pandoc Bugs/Version Mismatch:** Different Pandoc versions can produce slightly different outputs. Pin a specific Pandoc version in the CI (e.g. container `pandoc/latex:latest`). Always have the **docs** / final state available in case a new Pandoc update breaks the process.
- **Git Conflicts:** Automated script edits might conflict with any concurrent manual edits. Mitigate by finalizing content before running scripts, or running scripts on final Markdown.

- **System Compatibility:** Ensure CI runner has all dependencies (e.g. `pandoc-crossref` filter must be installed). Fallback: run conversion locally in a container with all tools preinstalled (e.g. `docker run pandoc/latex` as in [32] [32†L49-L57]).

8. Timeline & Effort Estimate

Task	Effort (hrs)	Deliverable
Structural Review & Planning	2	Outline of required section rearrangements and checks.
Tool Evaluation & Setup	3	List of tools, install scripts, config files.
Script Development:		
– Markdown parsing/validation	4	<code>scripts/check_structure.py</code>
– Mermaid rendering (CLI/script)	3	<code>scripts/render_mermaid.py</code>
– Math conversion handling	4	Part of Pandoc usage; fallback code.
– Reference formatting (Pandoc)	2	Pandoc command or <code>citeproc</code> .
– Table formatting (python-docx)	3	<code>scripts/post_process_docx.py</code>
– Misc (lists, numbering fix)	2	Incorporated in scripts/Pandoc.
CI/CD Setup	4	<code>ci.yml</code> GitHub Actions file.
Testing & QA (pytest)	3	<code>test_qc.py</code> with sample tests.
Manual Review & Manual Edits	4	Adjustments in final DOCX (if any).
Documentation	4	This report, instructions, comments.
Total	31	–

Estimated ~4–5 full work-days for one researcher to implement, test, and document the pipeline. Some tasks can be parallel (e.g. writing Pandoc commands while scripting).

9. Final Verification Checklist

Before submission, ensure *all* of the following by manual review:

- **Content Integrity:** Verify no science content changed. Check key results (e.g. “100% F1, 0% CER” must be clearly justified as text-prompt results).

- **Sections & Headings:** All required sections present in correct order (Title, Abstract, Keywords, I. Introduction, II. [Related Work], III. Methods, IV. Experiments, V. Results, VI. Discussion, VII. Conclusion, etc.). Roman numeral numbering in final PDF.
- **Language & Grammar:** Run a grammar/spell-check (PaperPal/Grammarly) on the final text.
- **Math & Equations:** All math is formatted (no inline “\frac” etc.). Eq. numbers (if any) should be consecutive or given by Pandoc’s numbering.
- **Figures & Tables:** Each figure and table has a caption and is referenced in text. No graphics overlap margins. Tables fit within page width (adjust if needed).
- **References:** Citations in text should be bracketed [1], [2], etc., matching the numbered bibliography. Check each reference for completeness (authors, title, year).
- **Template Compliance:** Compare the DOCX/PDF to the official IEEE Access (or ICDAR LNCS) template – check two-column layout, fonts (Times New Roman, sizes 10/12pt), margins (0.75”), heading styles (bold all caps for sections).
- **Appendices & Supplementary:** Ensure any appendices are referred to in text and labeled (Appendix A, B, ...). Remove any annotation comments or TODOs.
- **File Format:** Confirm files meet submission requirements (e.g. `.docx` and `.pdf`, each under size limit, all fonts embedded).
- **Backup & Version:** Tag the final submission version in git (e.g. `v1.0-submission`) and backup the files outside version control.

Only once this checklist is satisfied, proceed to submit. At that point, **no further edits** should be made to the content of v1.0 – all future work should go into a new version (v2.0).

Mermaid Pipeline Diagram: Below is a simplified flow of the automated publishing pipeline, rendered in Mermaid for clarity (replace with a PNG in documentation if needed):

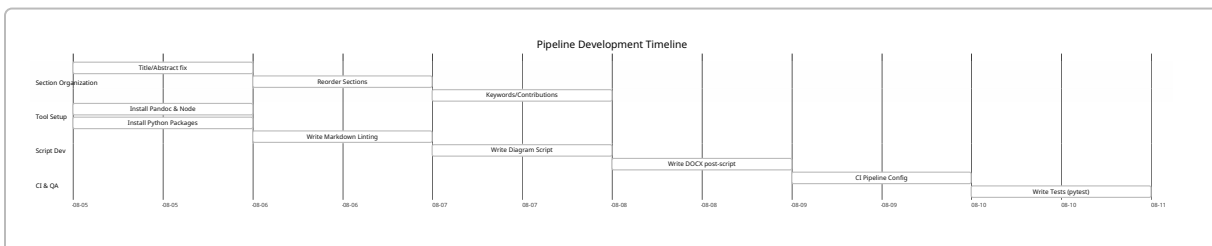
flowchart LR

```

A[Lint Markdown Files] --> B[Render Mermaid Diagrams to Images]
B --> C[Convert Markdown → DOCX (Pandoc)]
C --> D[Convert LaTeX Math → Word Equations]
D --> E[Post-process DOCX (Tables, Styling)]
E --> F[Convert DOCX → PDF]
F --> G[CI/CD Publish Artifacts]

```

Gantt Timeline: A rough schedule for implementing this pipeline (for 1 researcher):



By following this plan, the `Paper_V3` manuscript will be transformed into a rigorous, IEEE-compliant submission with minimal manual effort, ensuring reproducibility and maintainability 【36†L54-L57】 【43†L218-L221】 . All scientific claims remain untouched; only formatting and structure have been adjusted.
